ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №1

«Організація професійного Python-проєкту та Git»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Ознайомитися з принципами організації професійного Python-проєкту, створити ізольоване середовище, використати src-layout, модулі та пакети, pyproject.toml, Git і документацію. Реалізувати консольну систему обліку комп'ютерної техніки за варіантом №13.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — «Система обліку комп'ютерної техніки». Для кожного пристрою зберігати інвентарний номер, тип, виробника, модель і вартість. Реалізувати пошук, фільтрацію за виробником, обчислення середньої та загальної вартості обладнання, визначення найдорожчого пристрою.

Проєкт повинен мати src-layout, щонайменше три Python modules, data model, окремий module з business logic, main.py, type hints, README.md, .gitignore, pyproject.toml і Git repository.

3. Структура проєкту

```
computer-inventory/
├── pyproject.toml
├── README.md
├── .gitignore
├── docs/GITHUB.md
├── src/computer_inventory/
│   ├── __init__.py
│   ├── models.py
│   ├── services.py
│   └── main.py
└── tests/test_inventory.py
```

Каталог src/computer_inventory містить пакет застосунку. models.py визначає модель Device, services.py містить бізнес-логіку, main.py — консольний інтерфейс і точку входу. tests/test_inventory.py містить автоматизовані перевірки; docs/GITHUB.md — інструкцію публікації. Файл __init__.py позначає пакет та містить його версію.

4. Опис створених модулів

4.1. models.py

Пристрій представлено незмінюваним dataclass Device з п'ятьма полями. Метод __post_init__ видаляє зовнішні пробіли, відхиляє порожні текстові поля та некоректну вартість. Decimal забезпечує точні грошові обчислення. Допускаються суми від 0 до 10^{12} грн з точністю до копійок.

```
@dataclass(frozen=True)
class Device:
    """An inventory item; cost is a non-negative amount in UAH."""
```

```
    inventory_number: str
    device_type: str
    manufacturer: str
    model: str
    cost: Decimal

    def __post_init__(self) -> None:
        for field in (
            "inventory_number", "device_type", "manufacturer", "model",
        ):
            value = getattr(self, field).strip()
            if not value:
                raise ValueError("Текстові поля не можуть бути порожніми.")
            object.__setattr__(self, field, value)
        if (not self.cost.is_finite()
                or not 0 <= self.cost <= Decimal("1000000000000")):
            raise ValueError(
                "Вартість має бути числом від 0 до 1000000000000."
            )
        if self.cost != self.cost.quantize(Decimal("0.01")):
            raise ValueError("Допускається не більше двох знаків після коми.")
```

4.2. services.py

Реалізовано add_device(), search_devices(), filter_by_manufacturer(), total_cost(), average_cost(), most_expensive() і sort_by_cost(). Пошук перевіряє чотири текстові поля без урахування регістру, фільтр зіставляє повну назву виробника. Повторні інвентарні номери заборонено. Для порожнього списку сума й середнє дорівнюють нулю, найдорожчий пристрій — None. За рівної максимальної ціни обирається перший пристрій.

```
def search_devices(devices: list[Device], query: str) -> list[Device]:
    """Search a substring in all four text fields, ignoring letter case."""
    query = query.strip().casefold()
    if not query:
        return []
    return [d for d in devices if any(query in value.casefold() for value in (
        d.inventory_number, d.device_type, d.manufacturer, d.model,
    ))]


def filter_by_manufacturer(
    devices: list[Device], manufacturer: str,
) -> list[Device]:
    """Match a full manufacturer name, ignoring case and outer whitespace."""
    name = manufacturer.strip().casefold()
    return [d for d in devices if d.manufacturer.casefold() == name]
```

```
def total_cost(devices: list[Device]) -> Decimal:
    """Sum costs exactly, including an empty inventory."""
    return sum((d.cost for d in devices), Decimal("0.00"))


def average_cost(devices: list[Device]) -> Decimal:
    """Return a mean rounded to cents; empty inventory has mean zero."""
    if not devices:
        return Decimal("0.00")
    return (total_cost(devices) / len(devices)).quantize(
        CENT, rounding=ROUND_HALF_UP,
    )


def most_expensive(devices: list[Device]) -> Device | None:
    """Return the first maximal item, or None for an empty inventory."""
    return max(devices, key=lambda d: d.cost, default=None)
```

4.3. main.py

main.py відокремлює введення й виведення від обчислень. Без аргументів відкривається меню: додавання, список, пошук, виробник, статистика, сортування та вихід. Параметр --demo запускає відтворюваний приклад із чотирма пристроями. Перевіряються порожні поля, нечислові значення, від'ємні суми, NaN, Infinity та зайві десяткові знаки. Підтримано десяткову кому. Дані зберігаються у пам'яті протягом сеансу.

```
def main() -> None:
    parser = argparse.ArgumentParser(description="Облік комп'ютерної техніки")
    parser.add_argument(
        "--demo", action="store_true", help="Демонстрація варіанта 13",
    )
    args = parser.parse_args()
    try:
        if args.demo:
            run_demo()
        else:
            run_menu()
    except (EOFError, KeyboardInterrupt):
        print("\nРоботу завершено.")


if __name__ == "__main__":
    main()
```

4.4. test_inventory.py

Використано unittest зі стандартної бібліотеки Python. Десять тестів перевіряють розрахунки, пошук за всіма полями, точну фільтрацію, максимум і сортування, порожній список,

додавання та дублікати, некоректну модель, десяткові обчислення, повторне введення ціни й сценарій меню. Усі 10 тестів завершилися успішно.

5. Файл pyproject.toml

Секція [build-system] задає setuptools і механізм збірки. [project] містить назву, версію, опис, README, Python 3.11+ та порожній список зовнішніх залежностей. [project.scripts] створює команду computer-inventory, яка викликає main(). [tool.setuptools.packages.find] вказує шукати пакети в src. Editable-встановлення підключає вихідний код без перевстановлення після кожного редагування.

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "computer-inventory"
version = "0.1.0"
description = "Computer equipment inventory - Professional Python, variant 13"
readme = "README.md"
requires-python = ">=3.11"
dependencies = []

[project.scripts]
computer-inventory = "computer_inventory.main:main"

[tool.setuptools.packages.find]
where = ["src"]
```

6. Файл .gitignore

З Git виключено virtual environment, кеші, IDE configuration, build artifacts і .env.

```
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

7. Фрагмент README.md

```
# Облік комп’ютерної техніки
Лабораторна №1, варіант 13.
```

```
## Встановлення
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e .

## Запуск
.\.venv\Scripts\python.exe -m computer_inventory.main
.\.venv\Scripts\python.exe -m computer_inventory.main --demo

## Перевірка
.\.venv\Scripts\python.exe -m unittest discover -s tests -v

## Автор
Капарник Дмитро Андрійович, ФЕП-32
```

8. Створення virtual environment та запуск

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m computer_inventory.main --demo
computer-inventory --demo
python -m unittest discover -s tests -v
```

9. Результат роботи програми

Фактичне виведення демонстраційного запуску. Середовище перевірки: Python 3.12.14, Windows; пакет встановлено у режимі editable. За відсутності активації можна викликати .venv\Scripts\python.exe безпосередньо. У перевірці без доступу до індексу пакетів використано локальний setuptools і --no-build-isolation; для звичайного встановлення достатньо команд розділу 8.

```
ОБЛІК КОМП’ЮТЕРНОЇ ТЕХНІКИ — ВАРІАНТ 13
Усі пристрої:
INV-001 | Ноутбук | Lenovo | ThinkPad E14 | 32000.00 грн
INV-002 | Монітор | Dell | P2422H | 8500.00 грн
INV-003 | Ноутбук | Dell | Latitude 5440 | 45000.00 грн
INV-004 | Принтер | HP | LaserJet M110w | 7200.00 грн

Пошук: INV-002
INV-002 | Монітор | Dell | P2422H | 8500.00 грн

Виробник: Dell
INV-002 | Монітор | Dell | P2422H | 8500.00 грн
INV-003 | Ноутбук | Dell | Latitude 5440 | 45000.00 грн

Статистика:
Кількість пристроїв: 4
Загальна вартість: 92700.00 грн
```

```
Середня вартість: 23175.00 грн
Найдорожчий пристрій:
INV-003 | Ноутбук | Dell | Latitude 5440 | 45000.00 грн
```

10. Робота з Git

У локальному репозиторії створено три логічні коміти: структура і конфігурація; модель та бізнес-логіка; інтерфейс, перевірки й документація. Підготовлена історія має службового автора Lab Preparation. Перед власними змінами слід налаштувати user.name та user.email.

```
git init -b main
git add .gitignore pyproject.toml src/computer_inventory/__init__.py
git commit -m "Create project structure and packaging configuration"
git add src/computer_inventory/models.py src/computer_inventory/services.py
git commit -m "Add device model and inventory business logic"
git add src/computer_inventory/main.py tests README.md docs
git commit -m "Add console interface tests and documentation"
```

10.1. Результат git status

```
On branch main
nothing to commit, working tree clean
```

10.2. Результат git log --oneline

```
5c68ec0 Add console interface tests and documentation
5f6ad67 Add device model and inventory business logic
c5e4e9e Create project structure and packaging configuration
```

10.3. Перевірка результатів та експеримент

Для демонстраційних цін 32000, 8500, 45000 і 7200 грн отримано суму 92700 грн і середнє 92700 / 4 = 23175 грн. Максимум — INV-003 (Dell Latitude 5440), 45000 грн. Фільтр Dell повернув INV-002 та INV-003; пошук INV-002 — один монітор. Значення збігаються з ручним розрахунком.

Граничні перевірки: порожній список → 0.00 грн та відсутній максимум; пошук невідомого значення → порожній список; повторний номер → ValueError; від'ємна ціна, NaN, Infinity, 1.001 → відхилення. На сумах 0.10, 0.20, 0.20 грн отримано точну суму 0.50 грн і округлене середнє 0.17 грн (ROUND_HALF_UP). Команда unittest повідомила: Ran 10 tests, OK.

Кожна операція пошуку, фільтрації, суми й максимуму переглядає список за O(n). Сортування потребує O(n log n). Для ЛР №1 лінійний пошук відповідає невеликому набору даних; індекс за інвентарним номером можна дослідити в наступних роботах.

10.4. Завантаження на GitHub

Створити на GitHub порожній репозиторій computer-inventory без README та .gitignore. У каталозі проєкту вказати власні user.name і user.email. Після заміни YOUR_USERNAME виконати:

```
git remote add origin https://github.com/YOUR_USERNAME/computer-inventory.git
git push -u origin main
```

Автентифікацію завершити у браузері. Після push перевірити файли й історію на GitHub. Повна інструкція, зокрема випадок наявного origin, міститься в docs/GITHUB.md.

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-01, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

11. Відповіді на контрольні запитання

1. Що таке Python-проєкт?

Організована сукупність коду, конфігурації, документації, тестів та ресурсів програмного продукту.

2. Чим проєкт відрізняється від одного Python script?

Проєкт має модульну структуру, конфігурацію, документацію, тести та керування залежностями.

3. Що таке Python module?

Окремий файл .py, який містить Python-код.

4. Що таке Python package?

Каталог, що об’єднує пов’язані Python modules.

5. Для чого потрібний __init__.py?

Файл ініціалізації звичайного пакета: може задавати версію та публічний інтерфейс. Для namespace-пакетів __init__.py не обов’язковий.

6. Що таке src-layout?

Структура, у якій вихідний код розміщується в окремому каталозі src.

7. Які переваги має src-layout?

Чіткіша структура та коректніша перевірка встановленого пакета.

8. Для чого використовується virtual environment?

Для ізоляції залежностей проєкту.

9. Як створити .venv?

python -m venv .venv

10. Як активувати virtual environment?

Windows: .venv\Scripts\activate; Linux/macOS: source .venv/bin/activate.

11. Для чого використовується pip?

Для керування Python packages.

12. Чому рекомендується python -m pip?

Щоб використовувати pip саме поточного Python interpreter.

13. Для чого потрібний pyproject.toml?

Для метаданих, build system, залежностей і конфігурації інструментів.

14. Що містить секція [project]?

Назву, версію, опис, вимоги до Python, залежності та інші метадані.

15. Що таке editable installation?

Встановлення через pip install -e ., за якого редагування вихідних .py-файлів одразу використовується під час наступного запуску. Зміна метаданих або консольних команд може вимагати повторного встановлення.

16. Для чого використовується if __name__ == "__main__"?

Щоб точка входу виконувалася лише при безпосередньому запуску.

17. Що таке Git?

Розподілена система контролю версій.

18. Що таке repository?

Сховище файлів проєкту разом з Git-історією.

19. Що таке commit?

Зафіксований набір змін з унікальним ідентифікатором.

20. Для чого використовується git status?

Для перегляду стану working tree і staging area.

21. Для чого git add?

Для додавання змін до staging area.

22. Для чого git commit?

Для фіксації підготовлених змін.

23. Що показує git log?

Історію commits.

24. Для чого потрібен .gitignore?

Визначає шаблони файлів, які Git ігнорує, якщо вони ще не відстежуються. Додавання шаблону не прибирає файл, який уже потрапив до історії.

25. Чому .venv не потрібно комітити?

Воно велике, платформозалежне та легко відтворюється.

26. Для чого потрібний README?

Для опису призначення, встановлення та запуску проєкту.

27. Що таке type hints?

Анотації очікуваних типів параметрів, результатів і змінних. Вони допомагають IDE та статичним аналізаторам, але самі по собі не перевіряють типи під час виконання.

28. Для чого програму розділяють на modules?

Для модульності, повторного використання, тестування та супроводу.

29. Що таке business logic?

Правила та обчислення предметної області.

30. Чому потрібно відокремлювати application entry point від business logic?

Щоб логіку було легше тестувати, повторно використовувати та підключати до інших інтерфейсів.

12. Висновок

Створено консольну систему обліку комп'ютерної техніки для варіанта №13. Реалізовано модель Device, модуль бізнес-логіки, меню та демонстраційний запуск. Пошук, фільтрація за виробником, середня й загальна вартість та найдорожчий пристрій працюють на контрольному наборі. Додано валідацію, сортування, type hints і 10 успішних автоматизованих тестів. Проєкт має src-layout, pyproject.toml, .gitignore, README та історію Git з трьох логічних комітів. Ця структура є основою для наступних лабораторних робіт за тією самою предметною областю.